Algorithmic Problem Solving 2026

Project Report

Group	Group H
Members	Karl Thedor Ruby (krub@itu.dk) Kristoffer Mejborn Eliasson (krme@itu.dk) Lukas Shaghashvili-Johannessen (lush@itu.dk)
Supervisor	Holger Dell, Martin Aumüller
Course	Algorithmic Problem Solving (APS 2026)
Date	2026-05-22

IT University of Copenhagen

Contents

1. Abstract

This report documents the algorithmic problem solving work of Group H for APS 2026.

We designed **The Siege**, a problem combining two classical techniques: maximum network flow and 0/1 knapsack. Each of N strongholds is modelled as a directed flow network; the gold yield equals the max-flow from source to sink. A warrior budget B constrains which strongholds can be conquered. The intended solution runs Edmonds-Karp per stronghold and solves the resulting knapsack with dynamic programming in $O(N \cdot VE^2 + NB)$ time. A brute-force $O(2^N)$ subset search and a greedy ratio heuristic serve as the time-limit-exceeded and wrong-answer foils respectively.

We solved three problems each of which corresponds to one of the categories presented in the course. **Robert Hood** (Geometric Algorithms) finds the diameter of a point set using Andrew's monotone chain convex hull followed by rotating calipers in $O(n \log n)$. **Cookie Selection** (Algorithms on Arrays) maintains a dynamic multiset under insertions and median queries using a Fenwick tree with coordinate compression and binary descent in $O(n \log n)$. **Jagged Skyline** (randomized algorithms) uses Monte Carlo column sampling with binary search to find the tallest building in $O(k \log h)$ queries expected, where k is the number of promising columns.

2. Designed Problem — The Siege (Dynamic Programming)

The Kattis-style problem "The Siege" is a problem that combines two paradigms from the course curriculum: network flow (max-flow on a directed weighted graph) and dynamic programming (0/1 knapsack). The solution requires one max-flow computation per stronghold followed by a single knapsack DP over the resulting values. The problem statement for the issue is as follows.

```
You are Warchief Grukk, and today is a good day for war. Your Horde stands ready to
crush the Dwarven Underkingdom, N strongholds carved deep into the mountains, each
sitting on rich veins of gold.

Your goblin scouts (small, sneaky, and not terribly bright) have somehow wormed their
way into every stronghold through cracks and drain pipes the dwarves forgot to seal.
They came back clutching crumpled maps scratched onto hide scraps, giggling about
"shiny rocks." The maps are crude but surprisingly accurate: each one shows the
layout of tunnels inside a stronghold.

Deep inside each stronghold sits the Gold Vein (room 0), where the richest ore waits
to be mined. Once you conquer a stronghold, your goblin miners will dig out the gold
and load it into mine carts that run through the tunnel network to the stronghold
entrance (room V_i - 1), where your wolf-riders wait with wagons.

Each tunnel is one-way: mine carts can only travel in one direction through it,
carved that way to keep traffic from jamming. Each tunnel also has a weight limit:
the maximum kilograms of gold ore that a mine cart can carry through it per hour.
Your miners will haul as much gold as physically possible from the vein to the
entrance, using every tunnel to its limit simultaneously. You need to figure out how
much gold each stronghold can yield.

But strongholds do not fall for free. The dwarves fight like cornered badgers behind
their stone barricades. Each assault costs orc warriors their lives. You can muster
exactly B warriors for the entire campaign, and you must conquer your chosen
strongholds before the elven relief army arrives. Choose which strongholds to take to
maximise the total gold your miners extract. Each stronghold can only be taken once.
```

`Present your Siege Plans to the war council: the list of strongholds the Horde will conquer.`

The actual problem the contestant is tasked with is to compute the maximum total gold extractable from any subset of strongholds whose combined warrior cost does not exceed B, and to output the indices of that optimal subset in ascending order. We nominate **Dynamic Programming** as the curriculum topic this designed problem covers for exam-cancellation purposes.

2.1. Accepted Solutions

All intended solutions decompose the problem into two independent subproblems applied sequentially: first compute a per-stronghold gold value via max-flow, then select the optimal stronghold subset via 0/1 knapsack DP. The Python and Rust accepted submissions implement this decomposition identically; the Rust version is included as a fast reference.

2.1.1. Per-stronghold Max-flow

For each stronghold i, the maximum flow from room 0 (Gold Vein) to room $V_i - 1$ (entrance) is computed using **Edmonds-Karp**, a BFS-augmented variant of Ford-Fulkerson. BFS guarantees that each augmenting path is shortest in the residual graph, giving a running time of $O(V_i \cdot E_i^2)$ per stronghold. With the parameter bounds $V_i \leq 15$ and $E_i \leq 40$, this is at most $15 \cdot 40^2 = 24000$ residual-edge inspections per stronghold, and at most $N \cdot 24000 = 1.2 \cdot 10^6$ inspections across the whole input. The residual graph's reverse-edge handling is required to permit later corrections of previously committed flow.

2.1.2. 0/1 Knapsack DP

Once each stronghold has a gold value g_i and a cost c_i, the selection problem reduces to 0/1 knapsack. A two-dimensional bottom-up DP fills the table

$$\mathrm{dp}[i][j] = \begin{cases} \mathrm{dp}[i-1][j] & \text{if } \; j < c_i \\ \max(\mathrm{dp}[i-1][j],\; g_i + \mathrm{dp}[i-1][j-c_i]) & \text{otherwise} \end{cases}$$

with base case $\mathrm{dp}[0][j] = 0$. The chosen indices are recovered by backtracking from $\mathrm{dp}[N][B]$: stronghold i was selected iff $\mathrm{dp}[i][j] \neq \mathrm{dp}[i-1][j]$. DP runs in $O(N \cdot B) = 5 \cdot 10^4$ steps, and backtracking is $O(N)$.

2.1.3. Combined Running Time

The overall accepted solution runs in $O(N \cdot V_{\max} \cdot E_{\max}^2 + N \cdot B)$, dominated by the N max-flows for the given parameter bounds. Memory is $O(V_{\max}^2)$ for each adjacency matrix plus $O(N \cdot B)$ for the DP table.

2.2. Time Limit Exceeded Solutions

The TLE submission enumerates all 2^N subsets of strongholds, computes the total warrior cost and total gold for each feasible subset, and keeps the highest-gold subset within budget. It is correct on all inputs but runs in $O(2^N \cdot (N + V \cdot E^2))$, which exceeds the time limit for $N = 50$ where $2^{50} \approx 10^{15}$ iterations are required. The TLE submission doubles as an oracle for verifying the DP solution on small instances ($N \leq 20$).

2.3. Wrong Solutions

The WA submission sorts strongholds by the gold-per-warrior ratio $\frac{g_i}{c_i}$ in descending order and greedily adds strongholds while the warrior budget allows. This is the optimal strategy for **fractional** knapsack but fails on 0/1 knapsack whenever a high-ratio item blocks a more profitable lower-ratio

combination. A concrete counter-example with $B = 10$: items with $(c, g) = (4, 5), (4, 4), (6, 6)$ are taken by greedy in ratio order as $(4, 5) + (4, 4)$ for gold 9, whereas the optimal $(4, 5) + (6, 6)$ has gold 11. The WA submission therefore fails on any test case where the ratio ordering disagrees with the optimal subset; sample test `1-small-manual` is constructed to trigger this directly.

2.4. Input Generation

Input generation is handled by `generators/generator.py`, a parameterised random generator that takes a seed and the parameters $(N, B, V_{\max}, E_{\max}, w_{\max})$ and produces a syntactically valid problem instance. The generator chooses each stronghold's name from a fixed lowercase-letter dictionary, rejecting duplicates; picks each stronghold's warrior cost c_i uniformly in $[1, B]$; and builds each tunnel graph by sampling V_i and E_i within bounds and emitting random directed edges with weight in $[1, w_{\max}]$, rejecting duplicate (u, v) pairs and self-loops.

The bulk of the secret tests are produced by running this generator with different seeds across four scale tiers ($N = 1$, $N \leq 5$, $N \approx 20$, $N = 50$). Generated inputs are post-processed by the validator (see *Validation* below) which enforces uniqueness of the optimal subset and rejects test cases where two distinct subsets tie on gold.

2.4.1. Edge Case Inputs

A small number of secret tests were hand-crafted (or hand-edited after generation) to cover behaviours that random sampling rarely produces. A single-stronghold input ($N = 1$ with the lone stronghold within budget) exercises the smallest-input boundary. A zero-flow stronghold input contains one stronghold whose graph is disconnected from sink to source so its max-flow is 0; the DP must correctly treat it as a zero-value item. A cycle / back-edge input contains directed cycles that exercise Edmonds-Karp's residual-edge bookkeeping — a buggy implementation that omits reverse-edge updates under-counts flow here. A tie-break input was constructed so two subsets nearly tie on gold; the validator's uniqueness check rejects exact ties to keep the answer well-defined.

2.5. Parameters

Two parameters were chosen carefully; the remaining graph parameters scale with them. The stated bounds are $1 \leq N \leq 50$, $1 \leq B \leq 1000$, $2 \leq V_i \leq 15$, $1 \leq E_i \leq 40$.

2.5.1. Stronghold count N

N was capped at 50 to ensure that the brute-force submission (2^N) is decisively infeasible at the upper bound while remaining tractable for the DP submission at $O\big(N \cdot B + N \cdot V \cdot E^2\big)$ work. Lower values of N would allow brute-force to pass and weaken the TLE category; higher values would inflate the DP table beyond what the time limit comfortably permits in Python.

2.5.2. Warrior budget B

B was capped at 1000 so that the DP table size $N \cdot B = 5 \cdot 10^4$ is small enough to fit easily in memory and fast enough to fill in well under the time limit. Setting B higher would shift the bottleneck from the max-flow step to the DP, which is undesirable because the curriculum focus we want to emphasise (knapsack DP) is meant to be visible but not the run-time pain point.

2.5.3. Graph parameters V_i, E_i

The per-stronghold graph is intentionally tiny ($V_i \leq 15$, $E_i \leq 40$). This keeps each Edmonds-Karp run effectively constant-time and prevents the max-flow step from dominating run-time across $N = 50$ strongholds. Larger graphs would also weaken the curriculum framing by making the problem feel like a pure flow problem rather than a flow-plus-DP composition.

2.6. Validation

The input validator `input_validators/validator.py` enforces all syntactic constraints (regex-checked first line, name format and uniqueness, integer ranges for c_i, V_i, E_i, u, v, w, and absence of duplicate edges or self-loops within a stronghold), and then runs a full knapsack over the validated input to confirm that the optimal subset is **unique**. The validator exits with code 42 on success and 43 on either a syntactic violation or a tie on the optimal subset. This uniqueness check is what allows test inputs to specify a single canonical answer.

2.7. Verifyproblem

`verifyproblem` was run from the `problem/thesiege/` directory. All sample and secret test cases pass the validator (exit code 42). The accepted Python and Rust solutions pass all tests within the time limit; the brute-force submission is correctly flagged TLE on the $N = 50$ case and the greedy submission is correctly flagged WA on `1-small-manual`. No warnings were produced.

3. Solved Problems

3.1. Robert Hood (Geometric Algorithms)

3.1.1. Problem metadata

- **Problem name / Kattis link:** Robert Hood
- **Short formal I/O description:** Given n points in the plane with integer coordinates, output the maximum Euclidean distance between any two points, as a floating-point number.

3.1.2. Solution overview

The maximum distance between any two points in a finite set is always achieved by two points on the convex hull of the set. This reduces the problem to two steps: compute the convex hull, then find the diameter of the hull polygon.

The convex hull is computed using Andrew's monotone chain algorithm, which sorts the points lexicographically and constructs the lower and upper hull in two linear passes. The result is the vertices of the hull in counter-clockwise order.

The diameter of the convex polygon is then found using the rotating calipers technique. For each edge of the hull, the algorithm advances an antipodal pointer j as long as the next vertex is farther from the edge than the current one. The cross product of the edge direction and the vector between consecutive candidate points determines the direction to move j. Both endpoints of the current edge are checked against j at each step, so no antipodal pair is missed.

This approach was chosen because it achieves optimal asymptotic complexity and avoids the $O(m^2)$ brute-force over hull vertices, which could be slow when all input points lie on the hull.

3.1.3. Correctness argument

Reduction to hull. Any point strictly inside the convex hull cannot be a diameter endpoint, since projecting it outward to the hull boundary only increases distance. Therefore the maximum distance is realised by two hull vertices.

Rotating calipers. The algorithm maintains the invariant that j is the index of the vertex farthest from the directed edge between `hull[i]` and `hull[i+1]`, among all indices not yet "passed" by i. The cross product check `cx > 0` advances j when the next vertex is strictly farther, and stops otherwise. Because the hull is convex, the distance from an edge is unimodal over consecutive vertices, so the

pointer j never needs to retreat. Both `hull[i]` and `hull[(i+1) % m]` are compared to `hull[j]` at each step to cover all antipodal pairs correctly.

Edge cases. When $m = 1$ (all points identical) the distance is 0. When $m = 2$ (collinear or just two distinct points) the distance is computed directly. These are handled before the calipers loop.

3.1.4. Complexity and time-limit reasoning

Let n be the number of input points and m the number of hull vertices, with $m \leq n$.

- **Convex hull:** $O(n \log n)$ dominated by the initial sort.
- **Rotating calipers:** $O(m)$ since each pointer advances at most m times.
- **Overall:** $O(n \log n)$ time and $O(n)$ space.

For $n = 10^5$, this is approximately $10^5 \times 17 \approx 1.7 \times 10^6$ operations for the sort, plus a linear pass. The implementation avoids square roots until the final step by comparing squared distances, eliminating floating-point overhead from the inner loop.

3.1.5. Empirical worst-case testing

The worst case for the rotating calipers occurs when all input points lie on the convex hull, maximising m. A set of n points uniformly distributed on a large circle achieves this. The following generator was used:

```
import math
N = 100000
print(N)
for i in range(N):
    angle = 2 * math.pi * i / N
    x = round(1e9 * math.cos(angle))
    y = round(1e9 * math.sin(angle))
    print(x, y)
```

Running this input on the solution completed in approximately 0.73 seconds in CPython 3, which is within the Kattis time limit.

3.1.6. Implementation notes

- **Attributed to:** Lukas Shaghashvili-Johanesson (lush@itu.dk)
- **Language used:** Python 3
- **Files:** `solutions/geometry/roberthood.py`
- **How to run:** `python3 solutions/geometry/roberthood.py < input.txt`

3.2. Cookie Selection (Algorithms on Arrays)

3.2.1. Problem metadata

- **Problem name / Kattis link:** Cookie Selection
- **Short formal I/O description:** Given a sequence of up to 2×10^5 events, each either inserting a positive integer into a multiset or querying and removing the element at rank $\lfloor \frac{n}{2} \rfloor + 1$ (1-indexed) from the sorted multiset of current size n, output each queried value on its own line.

3.2.2. Solution overview

The core challenge is maintaining a dynamic multiset that supports two operations in sublinear time: insertion of an element and extraction of the element at a given rank. A naive sorted list achieves $O(n)$ per operation, which is too slow for $n = 2 \times 10^5$.

The solution uses a Fenwick tree over coordinate-compressed values. Because cookie diameters are up to 10^9 but there are at most 2×10^5 distinct values in the input, all values are mapped to contiguous indices $1, 2, ..., m$ where m is the number of distinct values. The BIT stores counts: position i holds how many cookies with compressed value i are currently in the holding area. A prefix sum query then gives the number of cookies with value at most i.

To answer a rank query for rank k, the solution uses a binary descent on the BIT structure, finding the smallest index i such that the prefix sum up to i is at least k. This descent runs in $O(\log m)$ rather than $O(\log^2 m)$ compared to binary searching over prefix sum queries.

This approach was selected because the BIT offers simple, cache-friendly implementation while achieving the necessary $O(\log n)$ per operation.

3.2.3. Correctness argument

Coordinate compression. All values are read before processing begins. The mapping $\text{compress} : v \mapsto i + 1$ is a bijection from the set of distinct input values to $\{1, ..., m\}$, preserving order. Thus rank in the compressed BIT equals rank in the original multiset.

BIT invariant. After each insertion of value v, the BIT count at position $\text{compress}(v)$ is incremented by one. After each removal, it is decremented by one. Therefore, at any point, `bit.query(i)` equals the number of cookies currently in the holding area with compressed index at most i, which equals the number of cookies with diameter at most $\text{decompress}(i)$.

Rank formula. The problem specifies that for n cookies the cookie sent is at position $\lfloor \frac{n}{2} \rfloor + 1$ in sorted order for both odd and even n. Setting `rank = n // 2 + 1` before each removal matches this specification exactly.

kth correctness. The binary descent in `kth(k)` maintains the invariant that `prefix_sum[1..pos] < k` throughout the loop and terminates with the smallest `pos + 1` satisfying `prefix_sum[1..pos+1] >= k`, which is exactly the k-th occupied position.

3.2.4. Complexity and time-limit reasoning

Let N be the number of input lines and $m \leq N$ the number of distinct cookie diameters.

- **Preprocessing:** sorting the distinct values takes $O(N \log N)$.
- **Per event:** each insertion and each rank query performs one BIT update or descent, each costing $O(\log m) = O(\log N)$.
- **Overall:** $O(N \log N)$ time and $O(N)$ space.

For $N = 2 \times 10^5$ and a typical time limit of 1-2 seconds, roughly $2 \times 10^5 \times 17 \approx 3.4 \times 10^6$ elementary operations are required, which is well within budget even in Python given the constant factor of the BIT operations.

3.2.5. Empirical worst-case testing

A worst-case input alternates insertions and removals to keep the holding area non-empty throughout. The following generator was used:

```
N = 100000
for i in range(1, N + 1):
    print(i)
for _ in range(N):
    print('#')
```

This produces 2×10^5 lines: 10^5 insertions followed by 10^5 queries, exercising both the coordinate compression and the BIT at maximum size. Running this input on the solution completed in approximately 0.67 seconds in CPython 3, which is within the Kattis time limit.

3.2.6. Implementation notes

- **Attributed to:** Karl Theodor Ruby (krub@itu.dk)
- **Language used:** Python 3
- **Files:** `solutions/arrays/cookieselection.py`
- **How to run:** `python3 solutions/arrays/cookieselection.py < input.txt`

3.3. Jagged Skyline (Randomized Algorithms)

3.3.1. Problem metadata

- **Problem name / Kattis link:** Jagged Skyline
- **Short formal I/O description:** Interactively discover the tallest building column in a skyline of width w and height h using height queries of the form "is there a building at column x of height at least y?". The interactor answers with a positive/negative response for each query.

3.3.2. Solution overview

- **Approach**: Monte Carlo sampling of columns combined with targeted binary searches. Randomly sample column order and probe increasingly higher heights only when a sample indicates promise; then binary-search that column's height.
- **Intuition**: Randomizing the search order avoids worst-case adversarial layouts and often finds a near-optimal column quickly; once a tall column is found, further work is limited because heights are bounded by h.

3.3.3. Correctness argument

- **Soundness**: Each confirmed positive query ("building" at (x, y)) is verified by binary search to determine the exact height of column x. The algorithm maintains the current best-known column and height and only replaces it upon finding a taller column.
- **Randomized benefit**: Random shuffle of columns yields good expected-time behavior against arbitrary distributions of tall columns; while not deterministic, the approach reduces the probability of scanning many low columns first.

3.3.4. Complexity and time-limit reasoning

- **Queries per sampled column**: In the worst case, a sampled column incurs one upward probe plus O(log h) queries for the binary search when successful.
- **Overall**: Expected number of probes depends on how many columns must be sampled before finding the global maximum; for typical inputs the randomized order finds the tallest column quickly, bounding total queries by roughly O(k log h) where k is the number of promising columns encountered.

3.3.5. Empirical behavior and robustness

- **Practical performance**: The implementation uses an early-exit when current best height reaches h. Randomization prevents pathological ordering and makes runtime stable across many test cases.
- **Robustness**: Works well in interactive judges since it never makes assumptions about skyline structure and always verifies heights before reporting the answer.

3.3.6. Implementation notes

- **Attributed to:** Kristoffer Mejborn Eliasson (krme@itu.dk)
- **Language used**: Python 3
- **Files**: solutions/random/jaggedskyline.py
- **How to run**: Run against the `testing_tool.py` script which is given in the the problem description on the [Kattis page](https://open.kattis.com/problems/jaggedskyline).

3.3.7. References

- **Source**: Monte Carlo sampling pattern implemented in solutions/random/jaggedskyline.py.